

RELATÓRIO DE AUDITORIA DE SEGURANÇA

Projecto VClass

30 de Agosto de 2026

Escopo Auditado

Repositório completo (backend Hono/Cloudflare Workers, 25 routers de API, middlewares de auth/CORS/rate-limit, 33 páginas HTML/JS do frontend, config. de deploy Cloudflare Pages/Workers, histórico Git)

Nota Metodológica

A stack do projecto foi detectada antes do início da auditoria: backend Hono sobre Cloudflare Workers/Pages, cliente Supabase (service_role, sem RLS activo), autenticação JWT própria (jsonwebtoken + bcryptjs), frontend em páginas HTML estáticas com JavaScript vanilla (sem framework SPA), deploy via wrangler.jsonc/Cloudflare Pages, CI em .github/. Cada categoria da auditoria foi mapeada ao mecanismo equivalente desta stack:

- Banco sem tranca (isolamento de tenant)

Não há RLS do Supabase em uso o cliente é sempre criado com a service_role key (bypassa RLS por desenho; ver src/config/supabase.ts). O isolamento é 100% responsabilidade manual de cada handler, filtrando por user_id / student_id / created_by / country_code. Auditados os filtros de todas as queries de listagem/agregação em 25 ficheiros de rotas.

- Permissão definida no navegador

Auth via JWT próprio (jsonwebtoken) + middleware requireRole(...) do Hono. Cruzados todos os gates de UI (isAdmin, role==="teacher", etc.) nas 33 páginas com o middleware equivalente no router Hono correspondente.

- IDOR

Percorridos sistematicamente os handlers .get/.post/.put/.patch/.delete com :id/:param dos 25 routers, verificando presença de .eq('<owner_column>', user.id) antes de devolver/alterar/apagar o recurso.

- Chaves expostas

Grep por padrões de chave (sk-, AKIA, key/secret/password/token literais) em src/, scripts, configs, wrangler.jsonc, .mcp.json, ecosystem.config.cjs e em todo o histórico git (git log -p); verificado .gitignore e ausência de .dev.vars real no repositório.

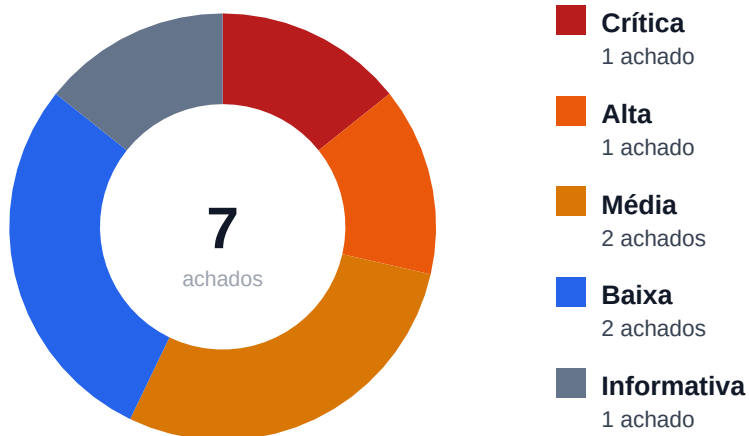
- Inputs sem tratamento (XSS)

Grep por innerHTML/insertAdjacentHTML/document.write nas 33 páginas HTML; verificado se o valor injectado passa por escapeHtml() antes do innerHTML; no backend, verificados os templates de email (utils/email.ts) quanto a interpolação directa de input do utilizador em HTML.

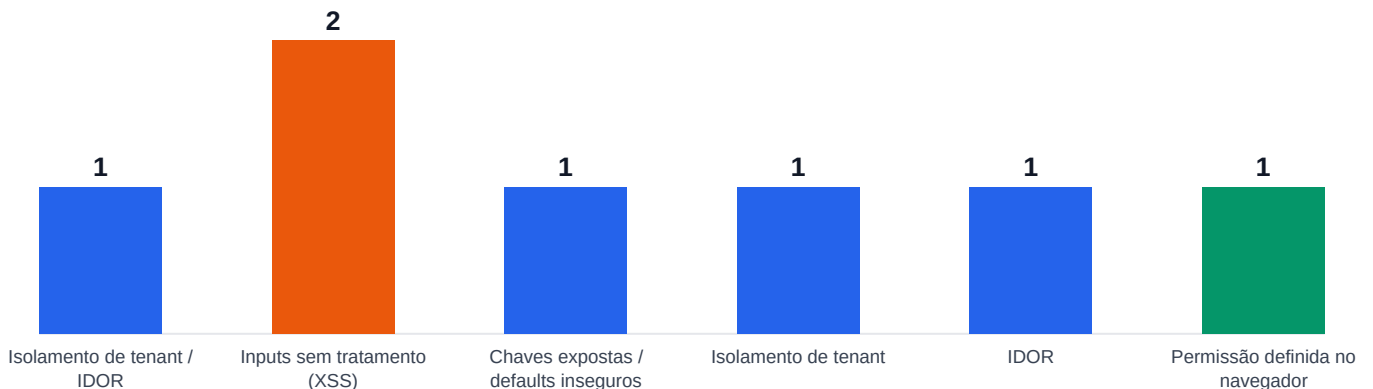
Resumo Executivo

A auditoria percorreu os 25 routers de API, os middlewares de autenticação/CORS/rate-limit, as 33 páginas do frontend e os ficheiros de configuração de deploy, verificando as cinco categorias solicitadas. Foram confirmados 7 achados (1 crítica, 1 alta, 2 médias, 2 baixas, 1 informativa) e documentados 8 pontos fortes com evidência concreta no código.

Achados por Severidade



Achados por Categoria



Pontos Fortes (protegidos, com evidência)

OK Isolamento de dados do próprio utilizador

progress.ts, favorites.ts, bookmarks.ts, comments.ts (DELETE), sessions.ts e exercises.ts filtram consistentemente por `.eq('student_id'|'user_id', user.id)` em cada query de leitura/escrita/remoção.

OK Posse verificada em conteúdo de professor

creator.ts confirma `.eq('created_by', user.id)` antes de editar/apagar lição, capítulo, sessão ao vivo e item de biblioteca em 10+ handlers distintos.

OK Isolamento de país replicado correctamente noutro módulo

teacher-verification.ts implementa `getManagerCountry()` e compara `app.country_id` com o país do gestor em 5 rotas (`/applications`, `/:id`, `/:id/review`, `/:id/approve`, `/:id/reject`, `/:id/request-info`) prova que o padrão correcto já existe no projecto, tornando a omissão em `country.ts` (F1) uma regressão isolada, não uma lacuna de arquitectura.

OK Verificação de papel no servidor para todas as rotas privilegiadas

`admin.ts`, `editor.ts`, `finance.ts`, `moderator.ts`, `support.ts` e `country.ts` aplicam `requireAdmin` / `requireEditorOrAdmin` / `requireFinanceOrAdmin` / `requireModeratorOrAdmin` / `requireSupportOrAdmin` / `requireCountryManagerOrAdmin` como middleware global do router (nunca dependem só da UI esconder botões).

OK Segredos obrigatórios sem fallback inseguro

`utils/jwt.ts` (`requireSecret`) e `routes/video.ts` (`getVideoSecret`) lançam erro em runtime se `JWT_SECRET/VIDEO_SECRET` não tiverem, respectivamente, 16/32 caracteres não existe nenhum valor por omissão que funcione como segredo real.

OK Nenhum segredo real no repositório ou histórico git

`.dev.vars` nunca foi commitado (confirmado via `git log --all -- .dev.vars` e `git ls-files`); `.gitignore` cobre `.env*/.dev.vars/.mcp.json`; `grep` por padrões de chave (`sk-`, `AKIA`, `key/secret` literais) no código-fonte não encontrou nenhum valor real, só placeholders (`sua-secret-super-secreta`, `your-...`) em ficheiros de exemplo.

OK Escaping consistente de conteúdo gerado por utilizador no frontend

`lesson.html` usa `escapeHtml()` em nome de autor, texto de comentário/resposta, título de marcador e conceitos-chave antes de qualquer `innerHTML` a excepção isolada é o Chat IA (F2).

OK Tokens de vídeo com escopo e expiração

`routes/video.ts` assina tokens HMAC-SHA256 com escopo (`userId × lessonId × videoId`) e expiração de 4h, válida IP/pertença ao utilizador antes de servir qualquer stream o URL real do CDN nunca chega ao browser.

OK CSRF mitigado por desenho

O access token vai em header `Authorization` (nunca em cookie), imune a CSRF clássico; o refresh token usa cookie `HttpOnly + Secure + SameSite=Strict` (`routes/auth.ts`).

Pontos Fracos (riscos centrais)

O risco central desta auditoria é a ausência de RLS no Supabase combinada com uma única lacuna de verificação de tenant em `country.ts` (F1): a arquitectura confia inteiramente em cada handler filtrar manualmente por dono/país, e uma única rota que esqueça esse filtro expõe dados de todos os tenants. O segundo risco central é a inconsistência de escaping no Chat IA (F2): o resto do frontend escapa HTML de forma disciplinada, mas o chat quebra esse padrão e permite roubo de token de sessão. Ambos são corrigíveis de forma cirúrgica (replicar padrões já existentes no próprio código), sem redesenho de arquitectura.

Achados Detalhados por Categoria

Isolamento de tenant / IDOR

Crítica**F1 Country Manager acede e altera dados de qualquer país, não apenas o seu**

src/routes/country.ts : 326-460

Todas as rotas /api/country/:id/* (stats, teachers, users, curriculum, teachers/:tid, announcement) usam apenas o middleware de papel requireCountryManagerOrAdmin (middleware/auth.ts:70), que valida SÓ o papel "country_manager" nunca compara o :id do país pedido com o country_code do próprio gestor. GET /api/country/me (linha 302-323) resolve correctamente o país do gestor a partir de users.country_code, mas nenhuma das rotas seguintes reaproveita essa verificação.

Código:

```
// src/routes/country.ts
country.use('/', authMiddleware)
country.use('/', requireCountryManagerOrAdmin) // só verifica o PAPEL

// GET /api/country/:id/users (linha 395)
country.get('/:id/users', async (c) => {
  const id = c.req.param('id') // <- vem directo do URL
  ...
  .eq('country_code', id) // nunca comparado com o país do gestor
})

// PATCH /api/country/:id/teachers/:tid (linha 445)
country.patch('/:id/teachers/:tid', async (c) => {
  const tid = c.req.param('tid')
  ...
  .update({ is_active: body.status !== 'inactive' }).eq('id', tid) // desactiva QUALQUER professor
```

Por que é explorável:

Um gestor de país (role=country_manager) atribuído a Angola tem um JWT válido para o seu papel. Nada nesse token ou nestas rotas restringe QUAL país ele pode gerir. Com o mesmo login, ele pode chamar GET /api/country/mz/users e obter nome+email+nota média de todos os alunos e professores de Moçambique, ou PATCH /api/country/mz/teachers/{id} para desactivar a conta de um professor moçambicano, ou POST /api/country/mz/announcement para enviar avisos em nome de outro país. O próprio código já resolve o "meu país" correctamente em outro ficheiro (src/routes/teacher-verification.ts, função getManagerCountry + comparação app.country_id !== managerCountry em 5 rotas) a omissão aqui é uma regressão/lacuna face ao padrão já estabelecido no mesmo projecto.

Condições de exploração:

Requer apenas uma conta válida de country_manager (papel de staff normal, não precisa de admin). Sem feature flag nem configuração insegura adicional reproduzível em qualquer ambiente com Supabase configurado.

Inputs sem tratamento (XSS)

Alta**F2 XSS reflectido no Chat IA permite roubo do token de sessão (localStorage)**

src/routes/chat.ts + src/pages/chat.html : chat.ts:195 · chat.html:313-320,402-411

Quando a pergunta do utilizador não bate com nenhuma tag da base de conhecimento local, o backend devolve a mensagem original do utilizador embutida sem qualquer escaping HTML dentro da resposta do "assistente". O frontend renderiza essa resposta com innerHTML após só converter marcações markdown (**negrito**, `código`), sem nunca escapar < > & antes.

Código:

```
// src/routes/chat.ts (fallback local, linha ~195)
return `Boa pergunta sobre **"${message.substring(0, 60)}"***! ...`
// 'message' é o texto CRU enviado pelo utilizador em POST /api/chat

// src/pages/chat.html (linha 402-411)
function formatBotText(text) {
  text = text.replace(/\*(.*?)\*/g, '<strong>$1</strong>') // NÃO escapa HTML antes
  ...
  return text
}
// linha 313-320
const formattedText = formatBotText(text);
div.innerHTML = `... <div class="text-sm">${formattedText}</div> ...` // injeção directa
```

Por que é explorável:

Enviar POST /api/chat {"message": ""} (uma frase que não corresponda a nenhuma tag da KB local o caso mais comum quando OPENAI_API_KEY não está configurada, que é o modo por omissão, ou sempre que a chamada à OpenAI falhar) faz o servidor devolver essa string HTML por dentro de "response", e o chat.html injecta-a directamente no DOM via innerHTML. O token de acesso (JWT) fica em localStorage.accessToken (confirmado em public/static/app.js:66,96,144) sem HttpOnly, plenamente legível por JavaScript. Um script injectado consegue portanto ler e exfiltrar esse token para um servidor externo, permitindo sequestro de sessão. Vector típico: engenharia social ("cola este texto no chat da VClass para desbloquear X") um "self-XSS" clássico que na prática rouba a própria sessão da vítima.

Condições de exploração:

Reprodutível sem nenhuma configuração especial: é o caminho por omissão sempre que OPENAI_API_KEY não está definida (variável opcional) ou a chamada à API OpenAI falha nesses casos o servidor cai sempre para este fallback local.

Baixa

F5 HTML de utilizador interpolado sem escape em emails transacionais

src/utils/email.ts : 73-130

Os templates de email (candidatura recebida/aprovada/rejeitada/info-pedida) interpolam directamente full_name (fornecido pelo próprio candidato em POST /api/teacher-verification/apply, sem autenticação) e a mensagem livre do avaliador, sem qualquer função de escape de HTML.

Código:

```
// src/utils/email.ts (linha 77)
html: emailLayout('Candidatura recebida ', `
  <p>Olá ${fullName}</p> // fullName vem directo do formulário público, sem sanitização
  ...`)
// linha 124 mensagem do avaliador também sem escape:
<p ...>${message}</p>
```

Por que é explorável:

Um candidato pode submeter full_name = "Clique aqui" (o schema Zod só valida comprimento mínimo, não caracteres) e esse HTML é injectado sem escape no corpo do email que o próprio Resend envia. A maioria dos clientes de email bloqueia <script>, mas HTML/CSS injectado pode alterar o layout ou introduzir links enganosos dentro de um email que parece legítimo da VClass vector menor de phishing/spoofing visual, não execução de script.

Condições de exploração:

Reprodutível sem autenticação (endpoint público /apply); impacto limitado porque o email é enviado apenas para o próprio endereço submetido pelo candidato.

Chaves expostas / defaults inseguros

Média

F3 Modo demo com password fixa e conta admin ficam activos se as variáveis do Supabase faltarem

src/middleware/database.ts + src/routes/auth.ts : database.ts:38-133 · auth.ts:247-284

Quando SUPABASE_URL ou SUPABASE_ANON_KEY não estão definidas no ambiente (env do Worker), o login (POST /api/auth/login) cai automaticamente em "modo demo": aceita qualquer email de mockUsers com a password fixa

DEMO_PASSWORD = 'vclass2024', incluindo a conta admin@vclass.mz com role='admin'. Não existe nenhuma validação de arranque que rejeite o deploy em produção sem estas variáveis configuradas.

Código:

```
// src/middleware/database.ts (linha 38)
export const DEMO_PASSWORD = 'vclass2024'
export const mockUsers = [
  ...
  { email: 'admin@vclass.mz', role: 'admin', ... }, // linha 120-133
]

// src/routes/auth.ts (linha 247)
if (!isDatabaseConfigured(c.env)) {
  const demoUser = mockUsers.find(u => u.email === email)
  if (!demoUser || password !== DEMO_PASSWORD) { ... 401 ... }
  // gera accessToken válido com role='admin' para admin@vclass.mz
```

Por que é explorável:

Isto corresponde exactamente ao padrão "default público que vira segredo real se não for sobrescrito": vclass2024 está em texto simples no repositório público. Se a env SUPABASE_URL/SUPABASE_ANON_KEY do Worker de produção não estiver definida (secret esquecido, nome trocado, binding não copiado ao criar novo ambiente Cloudflare Pages), a aplicação não falha de forma visível passa silenciosamente a aceitar login como admin@vclass.mz / vclass2024 com um JWT de admin válido, assinado com o JWT_SECRET real de produção. Não há nenhum "startup check" que impeça isto de acontecer num ambiente que se pensa estar em produção.

Condições de exploração:

Requer que as variáveis SUPABASE_URL/SUPABASE_ANON_KEY estejam ausentes nesse ambiente Cloudflare (configuração incorrecta) não é o caminho normal de produção documentado, mas nada no código impede ou avisa sobre esse estado.

Isolamento de tenant

Média

F4 Qualquer professor consegue listar nome e email de TODOS os alunos da plataforma

src/routes/creator.ts : 680-719

GET /api/creator/students devolve, para qualquer utilizador com role teacher/admin, a lista completa de utilizadores com role='student' e is_active=true (nome completo + email), de qualquer país, independentemente de o professor ter alguma vez leccionado a esse aluno. Só o progresso/nota é correctamente calculado apenas sobre as lições do próprio professor mas a listagem de identidade (nome+email) não é filtrada.

Código:

```
// src/routes/creator.ts (linha 703)
let usersQuery = supabase
  .from('users')
  .select('id, full_name, email, created_at', { count: 'exact' })
  .eq('role', 'student')
  .eq('is_active', true)
// <- sem filtro por país do professor nem por relação professor-aluno
```

Por que é explorável:

Um professor de Angola pode paginar /api/creator/students e obter nome+email de estudantes de Moçambique, Portugal ou Brasil com quem nunca teve qualquer interacção apenas porque a query busca TODA a tabela users filtrando só por papel. Isto viola o princípio de minimização de dados (o professor só precisa de ver os SEUS alunos) e expõe uma base de emails de milhares de estudantes a qualquer conta de professor.

Condições de exploração:

Reprodutível com qualquer conta de professor (role=teacher) real, sem configuração adicional; basta paginar ?page=N para exfiltrar toda a tabela de estudantes.

IDOR

Baixa

F6 Estado de processamento de vídeo Bunny acessível sem verificar posse

src/routes/creator.ts : 1124-1164

GET /api/creator/video/:videoid/status devolve título, thumbnail, duração e progresso de encoding de QUALQUER videoid do Bunny Stream da conta, sem confirmar que o vídeo pertence a uma lição criada pelo professor autenticado.

Código:

```
// src/routes/creator.ts (linha 1124)
creator.get('/video/:videoid/status', async (c) => {
  const videoId = c.req.param('videoid') // <- sem verificação de posse
  ... fetch(`https://video.bunnycdn.com/library/${bunnyLibraryId}/videos/${videoId}`, ...)
  return c.json({ success:true, data:{ title: v.title, thumbnailUrl: v.thumbnailUrl, ... } })
})
```

Por que é explorável:

Qualquer professor autenticado que adivinhe ou obtenha (ex: via inspecção de rede noutra aba) um videoid do Bunny de outro professor consegue ver o título e thumbnail desse vídeo antes mesmo de ser publicado. Impacto limitado (sem PII, sem acesso ao stream em si, que continua protegido por token HMAC em src/routes/video.ts), mas é uma verificação de posse em falta face ao padrão usado no resto do ficheiro (todas as outras rotas de creator.ts fazem .eq('created_by', user.id)).

Condições de exploração:

Requer conhecer um videoid válido (GUID do Bunny, não enumerável por sequência) reduz a probabilidade prática, mas o controlo de acesso está mesmo assim ausente.

Permissão definida no navegador

Informativa

F7 Páginas HTML de painéis privilegiados são servidas sem verificação de sessão

src/routes/pages.ts : 1-105

/admin-dashboard.html, /finance-dashboard.html, /country-dashboard.html, /moderator-dashboard.html, /editor-dashboard.html e /support-dashboard.html são servidas a qualquer visitante (pages.ts não aplica authMiddleware). Isto é apenas o "invólucro" estático (HTML/JS) todas as chamadas de API que essas páginas fazem para popular dados estão correctamente protegidas no backend (requireAdmin, requireFinanceOrAdmin, etc., confirmado nos routers correspondentes).

Código:

```
// src/routes/pages.ts sem authMiddleware em nenhuma rota
app.get('/admin-dashboard.html', (c) => c.html(adminDashboardHtml))
app.get('/finance-dashboard.html', (c) => c.html(financeDashboardHtml))
```

Por que é explorável:

Um visitante não autenticado consegue ver a estrutura/JS destes painéis (nomes de campos, endpoints chamados, lógica de negócio) sem autenticação, o que facilita o reconhecimento do alvo por um atacante mas não expõe dados reais, porque cada chamada de API subsequente é rejeitada com 401/403 pelo backend. Classificada como informativa por não haver fuga de dados, apenas de estrutura.

Condições de exploração:

Sempre acessível publicamente por desenho actual sem impacto directo de dados, apenas superfície de reconhecimento.

Recomendações Priorizadas

- P1** Corrigir country.ts (F1): adicionar a mesma verificação já usada em teacher-verification.ts obter o country_code do gestor autenticado e comparar com o :id do path em TODAS as rotas /api/country/:id/*, devolvendo 403 em caso de mismatch (admin continua sem restrição).
- P1** Corrigir a reflexão não escapada em chat.ts (F2): aplicar escape HTML ao texto do utilizador antes de o embutir em qualquer resposta textual, e/ou sanitizar no frontend (chat.html) antes do innerHTML nunca confiar em texto vindo da API para innerHTML sem passar por escapeHtml() ou um sanitizador (ex: DOMPurify).
- P2** Adicionar uma validação de arranque (fail-fast) que rejeite requests em produção se SUPABASE_URL/SUPABASE_ANON_KEY não estiverem definidas, em vez de cair silenciosamente em modo demo com password fixa (F3). Alternativamente, gerar a password demo aleatoriamente por deploy e nunca reutilizar "admin@vclass.mz" com uma password documentada publicamente.
- P2** Restringir GET /api/creator/students (F4) aos alunos com progresso registado nas lições do próprio professor (mesma lógica já usada para o cálculo de score), em vez de devolver toda a tabela de estudantes.
- P3** Escapar full_name/message antes de os interpolar nos templates HTML de email em utils/email.ts (F5) usar uma função simples de escape ou uma lib de sanitização de HTML de email.
- P3** Adicionar .eq('created_by', user.id) (via lookup da lição associada ao videoId) em GET /api/creator/video/:videoId/status (F6), alinhando com o padrão já usado no resto de creator.ts.
- P3** Opcional: aplicar authMiddleware/optionalAuth a pages.ts (F7) e devolver um redirect para /login.html quando a página exigir sessão, reduzindo a superfície de reconhecimento sem impacto funcional, já que os dados continuam protegidos pela API.

Issues para o GitHub

As issues abaixo estão prontas para copiar e colar directamente no GitHub. Cada bloco delimitado corresponde a uma issue completa em Markdown.

--- ISSUE 1 ---

Título: [Segurança] Country Manager acede e altera dados de qualquer país (crítico)

Labels sugeridas: security, critical

Problema

Todas as rotas `/api/country/:id/\*` (stats, teachers, users, curriculum, teachers/:tid, announcement) só validam o **papel** do utilizador (`requireCountryManagerOrAdmin`), nunca comparam o `:id` do país pedido com o `country\_code` do próprio gestor autenticado. `GET /api/country/me` resolve correctamente o país do gestor, mas as restantes rotas ignoram essa restrição.

Por que é explorável

Qualquer conta com `role='country\_manager'` (não precisa de admin) pode chamar estas rotas trocando o `:id` do URL por qualquer código de país, obtendo acesso total de gestão sobre países que não lhe pertencem.

Evidência

```
`src/routes/country.ts:395-411`
```ts
country.get('/:id/users', async (c) => {
const id = c.req.param('id') // vem directo do URL, sem verificação
...
.eq('country_code', id)
})
```

`src/routes/country.ts:445-460`
```ts
country.patch('/:id/teachers/:tid', async (c) => {
const tid = c.req.param('tid')
...
.update({ is_active: body.status !== 'inactive' }).eq('id', tid)
})
```
```

Impacto

- Leitura de PII (nome, email, notas) de alunos e professores de qualquer país.
- Desactivação/reactivação de contas de professores de outros países.
- Envio de avisos ("announcements") em nome de países que não gere.

Sugestão de correcção

Replicar o padrão já usado em `src/routes/teacher-verification.ts` (`getManagerCountry()` + comparação `app.country\_id !== managerCountry` 403) em todas as rotas `/api/country/:id/\*`, mantendo o admin sem restrição.

Critérios de aceite

- [] `GET/PATCH/POST /api/country/:id/\*` devolve 403 quando `role==='country\_manager'` e `:id` país do gestor.
- [] Admin continua a aceder sem restrição a qualquer `:id`.
- [] Teste automatizado cobrindo o caso de mismatch para cada rota afectada.
- [] `GET /api/country/me` continua a funcionar sem alterações.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] XSS reflectido no Chat IA expõe o token de sessão (alto)

Labels sugeridas: security, high

Problema

Quando a pergunta do utilizador não corresponde a nenhuma tag da base de conhecimento local, o backend devolve o texto do utilizador embutido sem escaping HTML na resposta do "assistente". O frontend injecta essa resposta via `innerHTML` depois de só converter marcações markdown, sem nunca escapar HTML.

Por que é explorável
Este é o caminho por omissão sempre que `OPENAI\_API\_KEY` não está configurada (variável opcional) ou a chamada à API OpenAI falha. O JWT de acesso vive em `localStorage.accessToken` (sem HttpOnly), por isso um script injectado consegue lê-lo e exfiltrá-lo.

```
## Evidência
`src/routes/chat.ts` (fallback local):
```ts
return `Boa pergunta sobre **"${message.substring(0, 60)}"***! ...`
```

`src/pages/chat.html:402-411`:
```js
function formatBotText(text) {
text = text.replace(/**(.*?)**/g, '$1') // sem escape prévio
return text
}
```

`src/pages/chat.html:313-320`:
```js
div.innerHTML = `... <div class="text-sm">${formattedText}</div> ...`
```
```

Impacto
Roubo do token de sessão (`localStorage.accessToken`) sequestro de conta. Vector prático: engenharia social ("cola este texto no chat para desbloquear X").

Sugestão de correcção
Escapar HTML do texto do utilizador antes de o embutir em qualquer resposta textual do backend, ****e**** sanitizar/escapar no frontend antes do `innerHTML` (nunca confiar apenas numa das duas camadas).

Critérios de aceite

- [] Enviar `- [] Testes cobrindo o fallback local e o caminho de erro da API OpenAI.
- [] Marcação markdown (negrito, código) continua a funcionar para texto legítimo.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Modo demo com password fixa pode conceder acesso de admin (médio)

Labels sugeridas: security, medium

Problema
Sem `SUPABASE\_URL`/`SUPABASE\_ANON\_KEY` configuradas, o login cai em modo demo: aceita qualquer conta de `mockUsers` (incluindo `admin@vclass.mz`, role `admin`) com a password fixa `DEMO\_PASSWORD = 'vclass2024'`, gerando um JWT real assinado com o `JWT\_SECRET` de produção. Não há validação de arranque que impeça isto.

Por que é explorável
Um erro de configuração (secret não copiado ao criar novo ambiente, nome de variável trocado) faz a app degradar silenciosamente para um estado onde qualquer pessoa que conheça a password pública do repositório (`vclass2024`) obtém um JWT de administrador válido.

```
## Evidência
`src/middleware/database.ts:38`
```ts
export const DEMO_PASSWORD = 'vclass2024'
```

`src/routes/auth.ts:247-256`
```ts
if (!isDatabaseConfigured(c.env)) {
const demoUser = mockUsers.find(u => u.email === email)
if (!demoUser || password !== DEMO_PASSWORD) { ... }
// gera accessToken válido com role do demoUser (inclui 'admin')
}
```
```

Impacto

Acesso administrativo completo caso o ambiente de produção fique, por engano, sem as variáveis Supabase configuradas.

Sugestão de correcção

Adicionar uma validação de arranque que recuse operar em modo demo fora de ambientes explicitamente marcados como dev/staging (ex: variável `ENVIRONMENT=production` obrigatória e incompatível com modo demo), ou gerar a password demo aleatoriamente por deploy.

Critérios de aceite

- [] Em produção, ausência de SUPABASE_URL/SUPABASE_ANON_KEY resulta em erro explícito, não em fallback silencioso para modo demo.
- [] Modo demo, quando permitido (dev/staging), documentado como tal na resposta da API.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] Lista de alunos exposta a qualquer professor sem filtro (médio)

Labels sugeridas: security, medium

Problema

`GET /api/creator/students` devolve nome completo e email de **todos** os utilizadores com `role='student'` e `is\_active=true` da plataforma, sem filtrar por país ou por relação professor-aluno. Só o progresso/nota é calculado apenas sobre as lições do próprio professor.

Por que é explorável

Qualquer conta de professor real (sem privilégio especial) consegue paginar este endpoint e exfiltrar nome+email de todos os estudantes da plataforma, incluindo os de outros países com quem nunca teve qualquer interacção.

Evidência

```
`src/routes/creator.ts:703-707`  
``ts  
let usersQuery = supabase  
.from('users')  
.select('id, full_name, email, created_at', { count: 'exact' })  
.eq('role', 'student')  
.eq('is_active', true)  
````
```

#### ## Impacto

Exposição em massa de PII (nome + email) de estudantes a qualquer professor, violando minimização de dados.

#### ## Sugestão de correcção

Restringir a lista aos alunos com progresso registado nas lições do próprio professor (mesma lógica já usada mais abaixo na função para calcular score/progresso), aplicando esse filtro **antes** da paginação, não só no cálculo de métricas.

#### ## Critérios de aceite

- [ ] `GET /api/creator/students` só devolve alunos com pelo menos um registo em `student\_progress` associado a uma lição do professor autenticado.
- [ ] Paginação e contagem total reflectem o novo filtro.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

### Título: [Segurança] HTML sem escape em emails transacionais e IDOR no estado de vídeo Bunny (baixo)

Labels sugeridas: security, low

Agrupados por serem de baixa severidade e afectarem áreas relacionadas de tratamento de input.

#### ## 1) HTML de utilizador não escapado em emails (`src/utils/email.ts:73-130`)

`full\_name` (submetido sem autenticação em `POST /api/teacher-verification/apply`) e a mensagem livre do avaliador são interpolados directamente em templates HTML de email, sem escape:

```
``ts
html: emailLayout('Candidatura recebida ', `
```

...

**\*\*Impacto:\*\*** injeção de HTML/links no corpo do email (phishing visual); a maioria dos clientes de email bloqueia `

## 2) IDOR no estado de vídeo Bunny (`src/routes/creator.ts:1124-1164`)

`GET /api/creator/video/:videoId/status` devolve título/thumbnail/progresso de qualquer `videoId`, sem confirmar que pertence a uma lição do professor autenticado todas as outras rotas do mesmo ficheiro fazem essa verificação via `.eq('created\_by', user.id)`.

## Sugestão de correcção

- Escapar `full\_name`/`message` antes de os interpolar em `utils/email.ts`.
- Adicionar verificação de posse (lookup da lição pelo `videoId`) em `GET /api/creator/video/:videoId/status`.

## Critérios de aceite

- [ ] Candidatura com `full\_name` contendo HTML não altera a estrutura do email enviado.
- [ ] `GET /api/creator/video/:videoId/status` devolve 404/403 quando o `videoId` não pertence a uma lição do professor autenticado.

--- FIM ISSUE 5 ---